

Celery + LLM Debug Guide (for Log Analysis & LLM Context)

1. Global Context

This system uses Celery workers with Redis as broker and result backend. Tasks trigger SQL queries, process data, and optionally call an LLM (Groq API) to generate structured analysis.

2. Key Components Observed in Logs

- Celery Worker: Executes async tasks
- Redis: Message broker and result store
- SQL Query Engine: Fetches banking data
- LLM (Groq / qwen model): Generates analysis from structured data

3. Execution Flow

1. Task received (lancer_traitement_restitution)
2. SQL query executed
3. Data prepared for frontend (list_champs_front)
4. LLM flag checked (var_llm = True)
5. Async LLM task triggered (lancer_llm_async)
6. Token estimation performed
7. API call to LLM (Groq)
8. Structured JSON analysis returned

4. Important Signals in Logs

- 'Task received': task execution start
- 'requete_sql': database query
- 'var_llm = True': LLM activation flag
- 'Tokens estimés': prompt size monitoring
- 'HTTP 200 OK': successful LLM call
- 'Analyse structurée reçue': final LLM output

5. Observed Behavior

Two parallel tasks are executed with identical queries and parameters. Both trigger LLM calls independently, producing slightly different outputs (normal behavior due to probabilistic models).

6. Potential Issues

- Duplicate task execution
- Data inconsistency (montent vs montant)
- Token overhead relatively high (516)
- Date format inconsistency (DD-MM-YYYY)
- Null values (montant = 0) affecting analysis

7. Optimization Recommendations

- Deduplicate tasks using unique IDs or locking
- Normalize schema (consistent field naming)
- Reduce prompt size to optimize token usage
- Validate data before sending to LLM
- Cache LLM responses if identical input

8. Why This Document is Useful for an LLM

This structured explanation helps an LLM understand:

- System architecture

- Execution flow
- Key signals in logs
- Known issues and improvements